

An Efficient Hybrid Exact String Matching Algorithm to Minimize the Number of Attempts and Character Comparisons

Prince Mahmud

Computer Science and Engineering
Discipline

Khulna University,

Khulna-9208, Bangladesh

princemahmud01740@gmail.com

Md. Sohel Rana

Computer Science and Engineering
Discipline

Khulna University,

Khulna-9208, Bangladesh

taj.sohelcse@gmail.com

Kamrul Hasan Talukder

Computer Science and Engineering
Discipline

Khulna University,

Khulna-9208, Bangladesh

khtalukder@gmail.com

Abstract—String matching fundamentally is a classical problem of finding occurrence(s) of a pattern string within another string or body of text. String matching problems can be traced into intrusion detection in network, detecting plagiarism, information security, pattern recognition, document matching, text mining, speech analysis, application in bioinformatics and other diversified fields. Two important factors of string matching which are also the challenges of this paper are “number of attempts” and “number of character comparisons”. With these challenges of string matching, we have proposed a hybrid algorithm which is named as MAC (Minimum number of Attempts and Character Comparisons) algorithm. We have integrated the concepts of Berry-Ravindran (BR) algorithm and index based shifting approach with our new search technique to build our MAC algorithm. We have evaluated the MAC algorithm to analyze the performance for English text alongside biological data (DNA sequence and Protein sequence). The performance of MAC algorithm has turned out to be better than Maximum-Shift (MS) algorithm and Index Based Shifting (IBS) algorithm. The performance of the MAC algorithm is proficient for exact string matching for both small and large size of pattern length comparing with some existing algorithm to solve the string matching problem.

Keywords-component— *Hybrid String Matching, Exact String Matching, Biological Sequence, English Text, etc.*

I. INTRODUCTION

String matching is a classical and traditional problem comprising some algorithms in computer science. Technically, these algorithms are known as string matching algorithms or sometimes called string searching algorithms. String matching algorithms are parts of an important set of string algorithms that try to find frequency (s) where one or several number of strings (also called patterns) are found within a large string or text. A string matching algorithm identifies an existence of a pattern string within a text string where strings are sequences of characters of some finite alphabet set. Let, a text of length n is $t = t_0, t_1, \dots, t_{n-1}$, a pattern of length m is $p = p_0, p_1, \dots, p_{m-1}$ and t_i, p_i are characters that belong to the alphabet set Σ . Note that, the length of the text is greater than or equal to the length of pattern (i.e. $n \geq m$). The aim of a string matching algorithm is to identify an existence of a

pattern p within a text t . The pattern and the search window are being compared character by character to find a pattern within a text string. ‘Search window’ is defined as the part of the text string which is compared with the pattern where the length of search window is equal to the pattern length [1].

String matching algorithms are mainly classified into two categories: exact string matching algorithm and approximate string matching algorithm [2]. In the process of exact string matching, the searching can be done by detecting exact number of occurrences of the pattern in the text. On the other hand, approximate string matching allows inaccurate searching with pattern in the given text. Exact string matching algorithm is categorized in two types: on-line and off-line. On-line string matching pre-processes the pattern and keeps the text intact and off-line string matching pre-processes the text [2]. On the other hand, string matching has two classifications based on the number of pattern in text; *Single Pattern string matching* and *Multiple Pattern string matching*. In case of single pattern string matching, only a single pattern is to be searched in the text. On the other hand, in multiple pattern string matching process, multiple patterns are searched in the text.

In this paper, we presented a hybrid exact string matching algorithm for bioinformatics and English text to minimize the number of attempts and character comparisons. The structure of the paper is as follows. Section II describes some existing exact string matching algorithms. Section III explains the proposed methodology. Section IV shows a working example. Section V analyzes and shows the experimental results and finally section VI concludes the proposed methods with some future direction.

II. RELATED WORK

In the field of computer science, there are many string matching algorithms concerned with how to slide the pattern or what strategy they use for pattern searching. Fundamentally, as a problem of string matching or pattern finding; the general technique (algorithm) that comes first to solve the problem is named as Brute Force algorithm also known as ‘naïve’ approaches. This is a common and oldest *Generate and Test* string matching algorithm [3]. In Brute Force algorithm, comparisons are performed character by character from left to right order. If the pattern and the search window do not match, then the pattern will be shifted only one position (all time) to the rightmost direction of the text. One of

the most popular and practical algorithm is Boyer More (BM) algorithm, which was proposed by Robert S. Boyer and J Strother Moore [4]. They introduced three searching ideas which were very smart enough to find a pattern. These are right-to-left comparison, bad character rule and good suffix rule. One of the most efficient variants of BM algorithm is Horspool algorithm which sacrifice the good suffix approach of BM algorithm. They use only the bad character rule to improve the performance of BM algorithm [5]. Another simplified version of BM algorithm is Quick search (QS) algorithm developed by adopting only the bad character rule with slight modification and avoiding the good suffix table [6]. QS algorithm constructs a bad character table in preprocessing phase and in searching phase, characters are compared from left to right direction. The algorithm shifts the pattern based on the next character of the right most character according to the bad character table. Another simplification of BM is Zhu-Takaoka algorithm which is enhanced the bad character shift function [7]. The main idea is centered in the preprocessing phase considering two characters to build the preprocessed table instead of using one character in the original BM algorithm. In the searching phase the comparisons are performed from right to left direction and when the mismatch occurs the pattern is positioned based on the two consecutive characters of the rightmost search window.

An efficient hybrid algorithm is Berry-Ravindran (BR) algorithm [6, 8]. The algorithm blends the good properties of original QS algorithm and the variant of Zhu-Takaoka algorithm. The main idea of BR algorithm is generating a preprocessed shifting function from the preprocessing function QS and Zhu-Takaoka (only bad character). In the searching phase, the pattern is shifted based on the shifting value of the two consecutive characters of text next to the search window and characters are compared from left to right direction. The recent hybrid algorithm of string matching is called Maximum Shift (MS) algorithm [1]. The algorithm integrates the effective properties of the Quick Search, Zhu-Takaoka and Horspool algorithm. In the searching phase at initial step, the right most characters of both search window and pattern are compared. If mismatch occurs, then the algorithm uses the QS preprocessed value based on the rightmost character of text next to the search window. If a match occurs, then the second last characters of both pattern and search window are compared and then left to right direction. If any match or mismatch occurs, then the algorithm uses the max-shift value between QS and Zhu-Takaoka shifting function. This algorithm is more efficient than the Quick Search, Horspool, Smith and Berry Rabindran algorithm for DNA sequence, protein sequence and English text.

A recent algorithm of string matching based on index based shifting approach is Index Based Shifting (IBS) algorithm [10]. The main idea of this algorithm is storing the index number of characters within a text as input, then computing substring based on the first character of pattern. The searching phase of the algorithm is almost same with Brute Forces approach except the first character comparisons. This

algorithm is more efficient than ABSBMH algorithm only for DNA sequence.

III. PROPOSED ALGORITHM

In our proposed algorithm, we integrate the concepts of Berry-Ravindran (BR) algorithm [6, 8] and Index based shifting approach [9, 10] with our new searching idea to minimize the number of attempts and character comparisons. The algorithm is divided into two phases; pre-processing phase and searching phase where the searching phase is a combination of maximum calculation and search technique.

A. Pre-processing Phase:

The pre-processing phase is based on the pre-processed table of BR algorithm and the index table. It should be noted that, the BR preprocesses the pattern and the index based shifting approach pre-processes the text. Our proposed algorithm generates three tables in the preprocessing phase. One is Berry Rabindran bad character table (*brBc*), Index table (*IT*) and according to our modification of index based shifting approach another is Index based shifting value (*IbSv*) table. To construct the *brBc* table, the shifting function of BR is:

$$brBc[a, b] = \min \begin{cases} 1 & \text{if } p[m-1] = a \\ m-i+1 & \text{if } p[i]p[i+1] = ab \\ m+1 & \text{if } p[0] = b \\ m+2 & \text{otherwise} \end{cases}$$

Here, p is the pattern and i is the index of pattern, a and b are two consecutive characters next to the search window. They must be the elements of alphabet set.

Next, Index table (*IT*) is easy to construct and the table stores the index number within the text string for each character belonging to the alphabet. However, the problem is which order we follow for character sequencing or ordering. For example, let the alphabet set $\Sigma = \{A, C, G, T\}$. To construct this index table, we use the ASCII value of the character. A character with small ASCII value will be selected first to figure out the index number of the character within the text. Using our idea, we sort the index table (*IT*) and construct the index based shifting value (*IbSv*) table only for first character of pattern. Note that, there is no need to store the index number in the *IbSv* table that exceeds the index number $n-m$

B. Searching Phase

The searching phase is a combination of two sub-phase; one is maximum calculation and other is search technique. In maximum calculation, our algorithm finds out the maximum value for pattern shifting. In addition, the algorithm searches the pattern within the text using our own search technique.

B-1. Maximum calculation:

To minimize the number of attempts and character comparisons our algorithm finds the maximum shifting value from *brBc* table and *IbSv* table. If the shifting value of *brBc* table for two consecutive characters next to the search window is maximum than *IbSv* table for first character of pattern, then

the MAC algorithm chooses the *brBc* shifting value and vice versa.

However, there is an important case to compute the *IbSv* shifting value. For computing the shifting value of *IbSv* table, we have to handle a pointer *i*. After aligning the pattern with search window, the pointer always points a value of *IbSv* table such that the value is greater than or equal to the index value *j* of first character of search window.

According to the Figure 1, in current attempt, pointer *i* points the value 1 in *IbSv* table and pattern is aligned with search window at position *j* = 1. At the end of this attempt, the pointer points the value 4 in *IbSv* table. Now assume that in the next attempt the pattern is shifted by the position *j* = 8 using *brBc* shifting value (Actually *IbSv* shifting value is 4-1 = 3 and we assume that the *brBc* shifting value 6). At the end of this attempt, the pointer points the value 5 in *IbSv* table. But this would be wrong! As *j* already points the index number 7 in search window, so we have to handle the pointer *i* such that the pointer points the value in *IbSv* table greater than or equal to *j* = 7. So, the *IbSv* value will be 8 where the value of pointer *i* is 3. In addition, the algorithm finds the *IbSv* shifting value by computing $IbSv[i] - j$.

Current Attempt:

$IbSv[i = 0] = 1$

a	1	4	5	8	9
---	---	---	---	---	---

$j = 1$

d	a	b	c	a	a	c	b	a	a	b	c	a	a
a	c	b	c	a									

$IbSv[i = 1] = 4$

a	1	4	5	8	9
---	---	---	---	---	---

Next Attempt:

$IbSv[i = 1] = 4$

a	1	4	5	8	9
---	---	---	---	---	---

$j = 7$

d	a	b	c	a	a	c	b	a	a	b	c	a	a
a	c	b	c	a									

$IbSv[i = 2] = 5$

a	1	4	5	8	9
---	---	---	---	---	---

✗

$IbSv[i = 3] = 8$

a	1	4	5	8	9
---	---	---	---	---	---

✓

Figure 1: *IbSv* shifting value calculation

B-2. Search technique:

In searching phase, the algorithm first aligns the pattern with searching window using maximum shifting value from *IbSv* table and *brBc* table. If index based shifting value is maximum, then there is no need to compare the first character of pattern with the first character of search window. Otherwise, the algorithm compares the first character. Then the algorithm compares the second character. If there is a match, then the algorithm compares the last one. And finally,

it compares the remaining character from left-to-right direction.

IV. WORKING EXAMPLE

The most efficient Maximum-Shift String Matching Algorithm [1] shows an example to search a pattern $p = GCAGAGAG$ with length 8 into a text t with length 47. This experiment was performed using the DNA sequence of chromosome I of *Arabidopsis thaliana* form index 32854 to 32901 [1]. The considering text t is

$t = ATCTAACATCATAACCCTAATTGGCAGAGAGAGAA$
TCAATCGAATCA

Now, we apply our proposed method on the text t for searching the pattern p . First, in the pre-processing phase, three shifting tables are processed which are Berry Ravindran bad character table, Index table and Index based shifting value table shown in Table 1, Table 2 and Table 3 respectively.

Table 1: Berry Ravindran Bad Character Table

	A	C	G	T
A	10	10	2	10
C	7	10	9	10
G	1	1	1	1
T	10	10	9	10

Table 2: Index Table

A	0	4	5	7	10	12	13	18	19	25
	27	29	31	33	34	37	38	42	43	46
C	2	6	9	14	15	16	24	36	40	45
G	22	23	26	28	30	32	41			
T	1	3	8	11	17	20	21	35	39	44

Table 3: Index Based Shifting Value (*IbSv*) Table

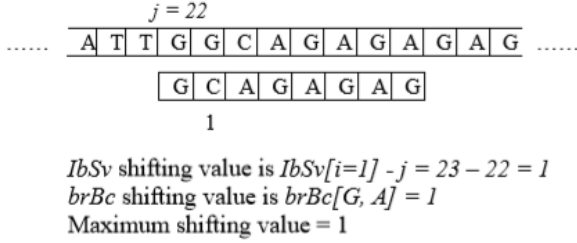
G	22	23	26	28	30	32
---	----	----	----	----	----	----

The Figure 2 shows the searching phase of our proposed algorithm. At the first attempt, the algorithm aligns the pattern with the text at $t[22]$ and the value of j (j points the index of text t) will be 22 and i will be 1 as initially it was 0 (i points the value of *IbSv* table). Initially there is no need to compare the first character. As the second character is mismatched the algorithm takes the maximum shift value. The indexing shift value is $IbSv[i] - 22 = 1$. And the BR max-shifting value is $brBc[G, A] = 1$. Here, A and G are two consecutive characters next to the search window. So, the algorithm can choose any of these and j will be 23 and i will be 2. In the next attempt, all characters are matched. And the shifting value of $IbSv[i=2] - 23 = 3$ is greater than shifting value of $brBc[A, G] = 2$. Using this shifting value (3) the algorithm aligns the pattern with search window at index number $j = 26$. Now the value of i is 3 that points the value 28 in *IbSv* table.

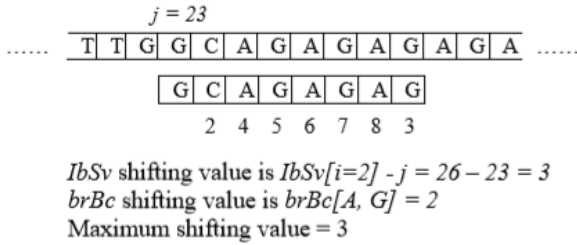
In the 3rd attempt, as the algorithm shifts the pattern using *IbSv* table then, there is no need to compare the first character. In this attempt, second character is mismatched and the shifting

value will be ($brBc[A, T] = 10$) greater than $IbSv$ shifting value. So, the algorithm aligns the pattern with search window at index number $j = 36$. But 36 is greater than the last value of $IbSv$ table (32). So, the algorithm terminates the procedure. However, the algorithm matches pattern within 3 attempts and 9 characters' comparison.

Initial Attempt:



2nd Attempt:



3rd Attempt:

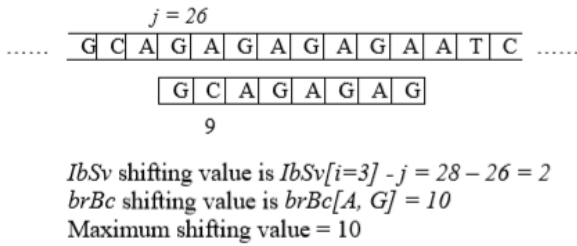


Figure 2: Searching Phase of Proposed MAC Algorithm

V. EXPERIMENTAL RESULT

A personal laptop with a processor Core i5 2.4 GHz and 4 GB RAM is used to perform all the experiment with C programming language after implementation. We tested and evaluated our proposed MAC algorithm using three different types of data. These types are DNA sequence with small alphabet set containing four characters, Proteins sequence with medium alphabet set containing twenty characters and English text with large alphabet set containing more than hundred characters. Note that, the size of each data type that is used to evaluate the algorithm is 200 MB. All the datasets are collected from the website named "Pizza&Chili" [2, 11]. Two important factors to measure the performance of the string matching algorithm is number of attempts and number of character comparisons. To evaluate the algorithm by these factors, we tested our algorithms for different length of

pattern. All patterns were selected randomly with length of 3, 5, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 150, and 200 within the input string/text.

This section shows the corresponding attempts and character comparisons for each length of pattern using 3 different types of data and compares the experimental results of proposed MAC algorithm with IBS (only for DNA sequence) and MS string matching algorithm. Noted that IBS algorithm is better for DNA sequences only, but not for English text and Protein sequence, where MS algorithm is better for all data types. Figure 3, 4 and 5 show the results of number of attempts for DNA sequence, protein sequence and English text respectively, where Figure 6, 7 and 8 show results for number of character comparisons. From these figures we may conclude the following measurement.

A. Number of Character Comparisons:

This factor is actually an important measurement to evaluate the performance of an algorithm. Moreover, the algorithm minimizes the number of character comparisons of search window and pattern that turns out the MAC algorithm to be better.

B. Number of attempts:

The MAC algorithm minimizes the number of this factor. Obviously, whenever the number of attempts is less, the algorithm will be proficient.

Finally, the MAC algorithm shows better results for IBS and MS algorithm for both large and small pattern using DNA sequence, protein sequence, and English text of size 200 MB. In addition, the IBS algorithm is better than ABSBMH [10] and MS algorithm is better than QS [6], Horspool [5], Smith [12] and BR algorithm [6, 8].

Though our algorithm is better comparing some existing algorithms, it has some limitations in some cases. If a pattern is such that $p = aaa$ and text is like $t = aaaaaaaaaaaaaaaaaa$ then our algorithm requires the maximum number of attempts and character comparisons. In this case, the searching time complexity becomes $O(mn)$. In addition, for best case ($p = bbb$, $t = aaaaaaaaaa$) the time complexity in searching phase is $O(1)$.

V. CONCLUSION

The MAC algorithm as a hybrid exact string matching algorithm, demonstrates better results than MS and IBS algorithms. The main reason behind the outcomes is that in the preprocessing phase of MAC algorithm - the shifting value increases the shifting distance that minimizes the number of attempts. In addition, the technique of searching phase minimizes the number of character comparisons. Finally, we can say that our algorithm shows better performance that minimizes the number of attempts and number of character comparisons for both large and small pattern using DNA sequence, Protein sequence and English text. The future target is to work on the time complexity. Another target of our work is to make the algorithm more efficient that may be better than this proposed algorithm.

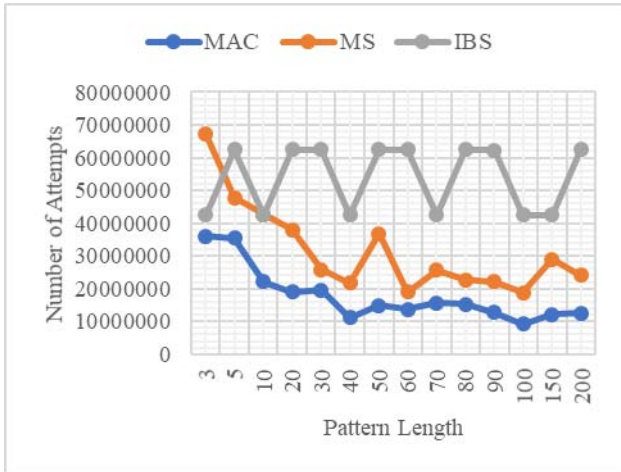


Figure 3: Number of attempts using DNA sequence.

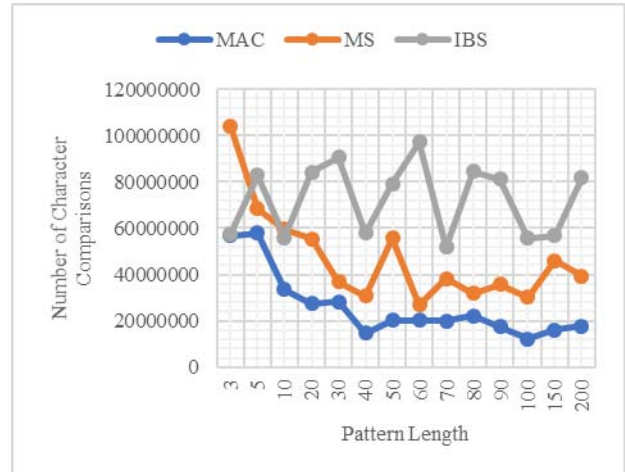


Figure 6: Number of character comparisons using DNA sequence.

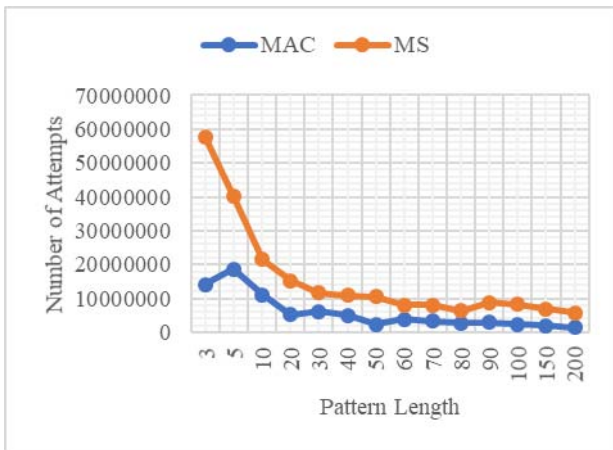


Figure 4: Number of Attempts using Protein sequence

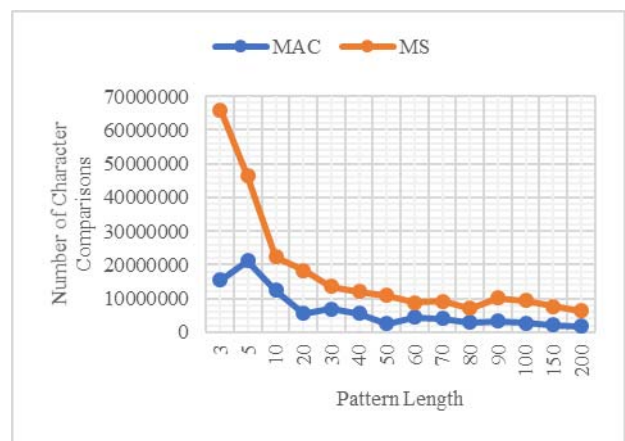


Figure 7: Number of Character Comparisons using Protein sequence

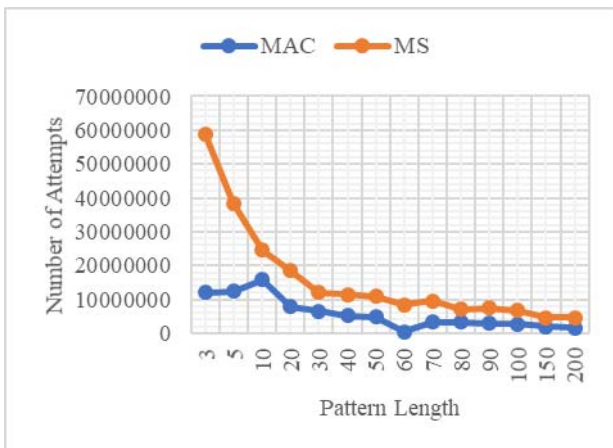


Figure 5: Number of Attempts using English Text

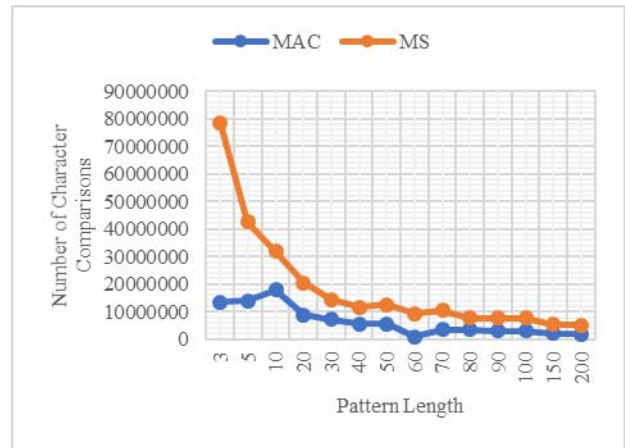


Figure 8: Number of Character Comparisons using English Text

REFERENCES

- [1] Kadhim, Hakem Adil, and NurAini AbdulRashid. "Maximum-shift string matching algorithms." *Computer and Information Sciences (ICCOINS), 2014 International Conference on*. IEEE, 2014.
- [2] Al-Dabbagh, Sinan Sameer Mahmood, and Nawaf Hazim Barnouti. "A New Efficient Hybrid String Matching Algorithm to Solve the Exact String Matching Problem." *British Journal of Mathematics and Computer Science*, pp. 1-14, 2017.
- [3] https://en.wikipedia.org/wiki/Bruteforce_search, accessed Aug, 2018.
- [4] Boyer, Robert S., and J. Strother Moore. "A fast string searching algorithm." *Communications of the ACM* 20.10 (1977): 762-772.
- [5] Horspool, R. Nigel. "Practical fast searching in strings." *Software: Practice and Experience* 10.6 (1980): 501-506.
- [6] Charras Christian, Thierry Lecroq. "Handbook of exact string matching algorithms." King's College; 2004.
- [7] Feng, Zhu Rui, and Tadao Takaoka. "On improving the average case of the Boyer- Moore string matching algorithm." *Journal of Information Processing* 10.3 (1987): 173-177.
- [8] <http://www-igm.univ-mlv.fr/~lecroq/string/berryravindran.html>, accessed Aug, 2018.
- [9] Bhukya, Raju, and D. V. L. N. Somayajulu. "An Index Based Sequential Multiple Pattern Matching Algorithm Using Least Count." *International Conference on Life Science and Technology IPCBEE*. Vol. 3. 2011.
- [10] Islam, Tania, and Talukder, Kamrul Hasan. "An improved algorithm for string matching using index based shifting approach." *Computer and Information Technology (ICCIT), 2017 20th International Conference of*. IEEE, 2017
- [11] https://www.genomatix.de/online_help/help/sequence_formats.html, accessed Aug, 2018.
- [12] Smith, P. D. "Experiments with a very fast substring search algorithm." *Software: Practice and Experience* 21.10 (1991): 1065-1074.